Chapter 1

Introduction to Graphs

1.1 Introduction

Some data is easier to work with if we imagine it as a set of connections. When we say *graph* in this chapter, we are referring to set of data defined by a collection of *nodes* (you may also hear vertex) and connected by *edges*. For example, on some social networks, each user can follow any number of other users. We can think of each user as a node, and the edge points from the user who follows to the user they follow:

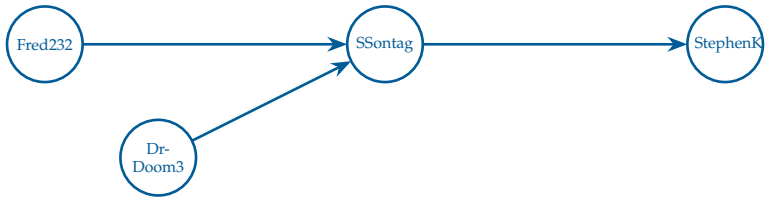


Figure 1.1: A graph showing four users and three "following" relationships.

Following is a *directed* relationship: Fred232 follows SSontag, but SSontag doesn't follow Fred232. So we would say that this is a *directed graph* with four nodes and three edges.

There are also undirected graphs. For example, you can imagine a graph that represents big data lines between cities. All the big data lines allow communications in both directions, like two-way roads:

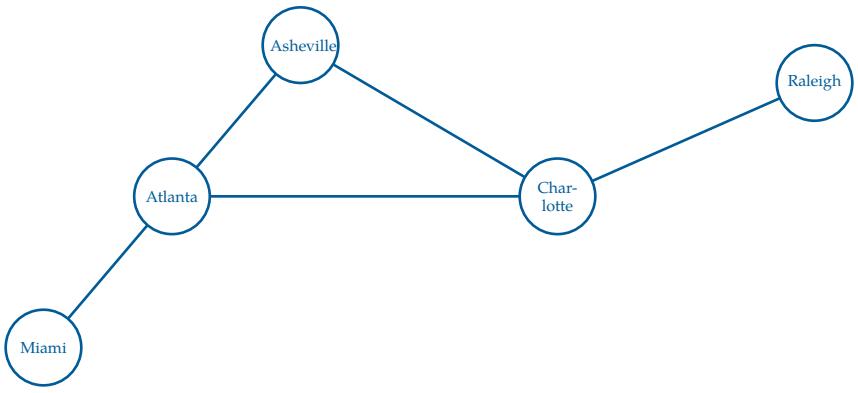


Figure 1.2: A set of cities, represented by nodes, on the east coast of the United States connected by roads, represented by edges.

The arrows are gone; if data can flow from Miami to Raleigh, then data can flow from Raleigh to Miama. A linked connection of edges between two or more nodes is called a *path*. Notice how Atlanta and Charlotte have 3 edges; we would say that Atlanta and Charlotte each have a *degree* of 3.

There is a whole branch of mathematics called *Graph Theory* that studies the properties of graphs. Here are two questions that mathematicians might ask about this graph:

- What is the smallest number of edges that we would need to follow to get from Miami to Raleigh?
- Does the graph have any paths where you could end up where you started?

That second question asks if there is a *cycle* in the graph. This graph has one cycle: Atlanta → Asheville → Charlotte → Atlanta.

Some graphs are *connected*: You can get from one node to any other node by following edges. Is this graph connected?

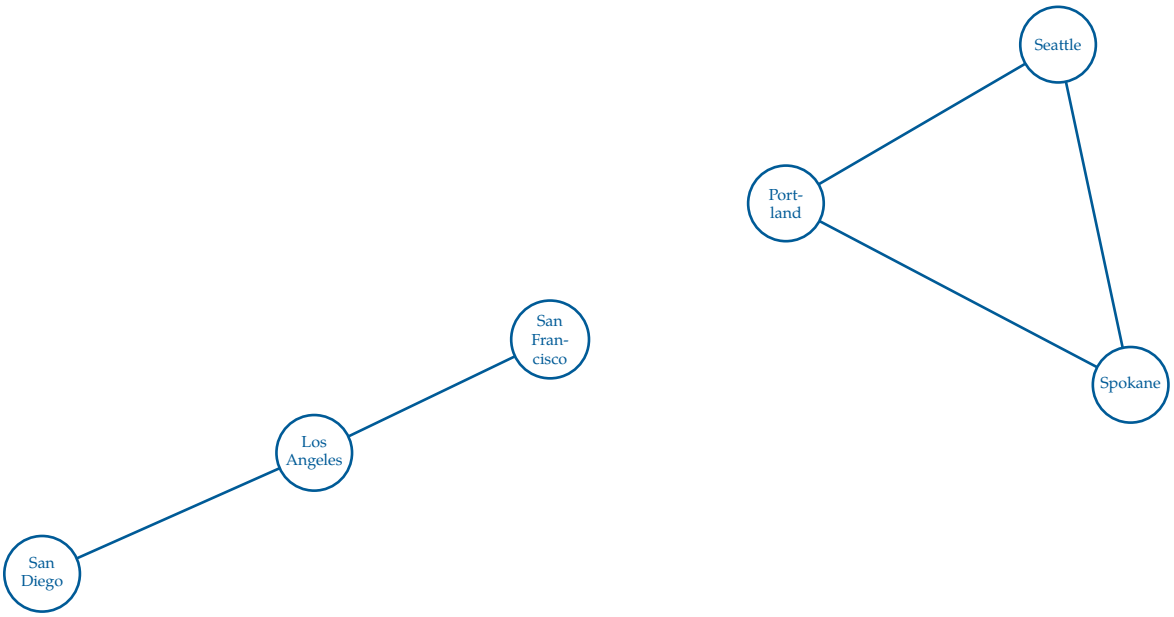


Figure 1.3: A set of cities (nodes) that are not all connected. Each set of *connected* cities forms a *connected component*.

This graph is *not* connected! You can't follow edges from San Diego to Seattle. However, you could identify two *connected components*, *subgraphs* of a graph that remain connected.

- San Diego, Los Angeles, San Francisco
- Portland, Seattle, Spokane

Connectivity is defined as, for all pairs of nodes in that graph, there exists a path between them. A subgraph is much simpler than a connected component, it's only requirement is to contain a subset of at least one node and, optionally, edges to go between nodes. Note that you cannot have a subgraph that contains a single node and an edge, because the edge would not be able to connect to anything else.

In graph data, the nodes and edges often have attributes. For example, a node representing a city might have a name and a population. An edge representing a data line might have a bandwidth (bits per second) and a latency (how many nanoseconds between when you put a bit into the pipe and when it comes out the other end). This is how we define a graph.

Exercise 1 Word Graphs

Working Space

Consider the concept of a graph as a set of nodes and edges. How would you create a graph where the nodes are words and the edges represent a one letter difference between two words?

For example, the word `cat` would be connected to `cot`, `cart`, and `cut`, but not to `dog`.

Define explicitly what the nodes would be in this graph, and what the edges would be. Draw a simple example of this graph with at least 5 nodes and 5 edges.

Answer on Page 9

1.2 Finding Good Paths

For many problems, we are trying to find the best path from one node to another. If all the edges are the same, this usually means finding the path that requires walking the fewest edges.

Sometimes the edges have a cost attribute. For example, you might want to find the

cheapest way to ship a container from New York City to Long Beach, California. In this case the nodes are train depots. Each train line between the depots has a cost. What is the cheapest path?

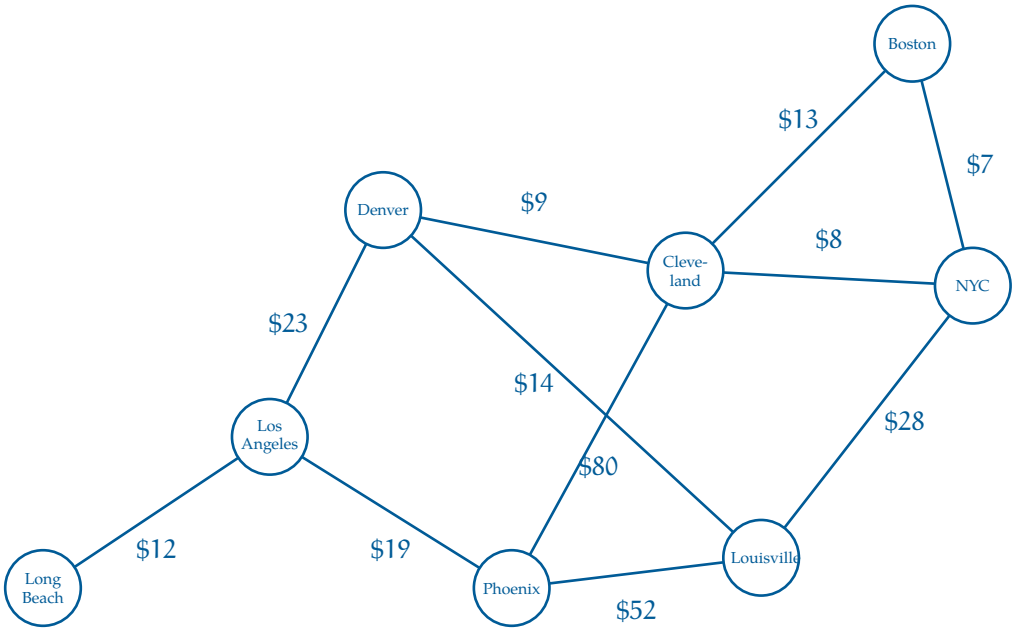


Figure 1.5: A graph of interconnected cities where each edge has a respective cost.

When edges have costs like this, we call the *weighted edges*, which forms a *weighted graph*. In weighted graphs, the *degree* of a node is split into the *in-degree* and *out-degree* of nodes. The in-degree identifies the incoming edges that originate from other nodes, and out-degree is the amount of edges going out of that node.

The graphs that you see here are really small, so finding efficient paths isn't difficult — you could just try all of them! However, in many computer programs, we are working with *millions* of nodes and edges. Efficient graph algorithms are *really* important.

1.3 Graphs in Python

There are many ways to represent graph data. There are database systems that are specifically designed to hold and analyze graph data. Not surprisingly, these are called *Graph Databases*.

You may also hear about *adjacency lists*, which store, for each node, the list of nodes it is connected to. Each direct connection from one node to another is called a *neighbor*. Simply, adjacency lists store lists of neighbors.

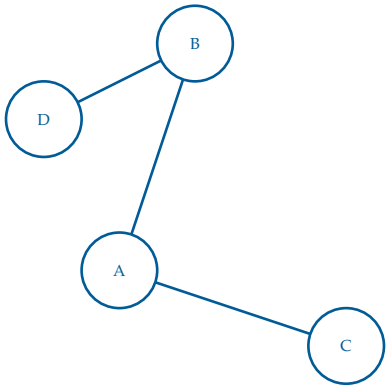


Figure 1.6: A simple (undirected unweighted) graph used to make an adjacency list.

As an adjacency list, this would look like:

- A: [B, C]
- B: [A, D]
- C: [A]
- D: [B]

Exercise 2 Directed Adjacency Lists

Working Space

Consider the concept of adjacency lists discussed above. How would the storage method change if the edges were to have weights?

Answer on Page 9

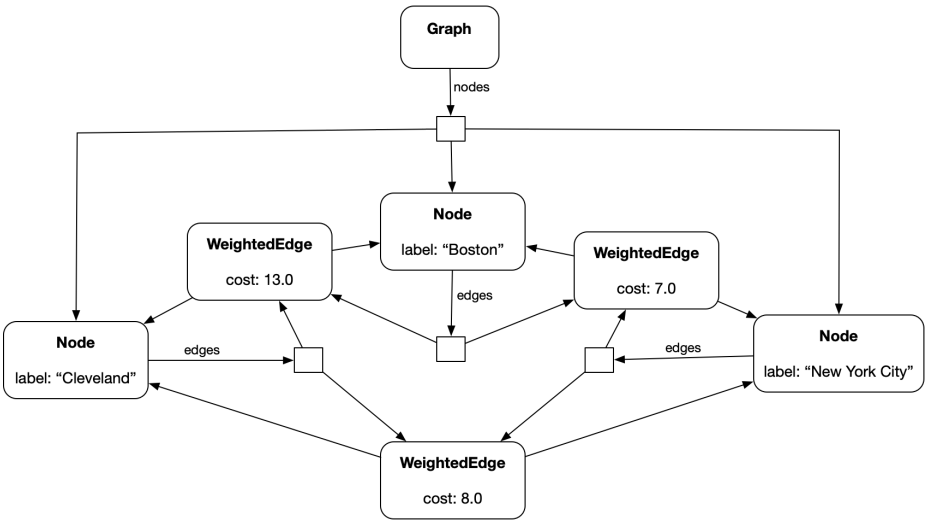
For simplicity, you are going to write Python classes that will let you represent an undirected graph with weighted edges, like the shipping problem above.

(Naturally, things would look a little different if the graph were directed or the edges were unweighted, but this is a good starting place.)

Create a file called `graph.py`. This will hold the code for your `Node` and `WeightedEdge` classes. We will also create a `Graph` class that will just hold onto the list of its nodes.

- A `Node` will have a label string and a list of edges that touch it.
- A `WeightedEdge` will have a cost and two nodes: `node_a` and `node_b`.
- A `Graph` will have a list of nodes.

Here is what the object diagram would look like if you had only three cities:



Put this code into `graph.py`

```
class Node:
  def __init__(self, label):
    self.label = label
    self.edges = []

  def __repr__(self):
    return f"(node:{self.label}, edges:{len(self.edges)})"

class WeightedEdge:
  def __init__(self, cost, node_a, node_b):
    self.cost = cost
    self.node_a = node_a
    node_a.edges.append(self)
    self.node_b = node_b
    node_b.edges.append(self)

```

```
def other_end(self, node_from):
    if self.node_a == node_from:
        return self.node_b
    if self.node_b == node_from:
        return self.node_a
    raise ValueError("node is not an endpoint of this edge")

class Graph:
  def __init__(self):
    self.nodes = []

  def add_node(self, new_node):
    self.nodes.append(new_node)

  def __repr__(self):
    return f"(Graph:{self.nodes})"
```

This is a graph class that handles undirected weighted graphs. Now, let's create some instances of `Node` and `WeightedEdge` and wire them together. Create another file in the same directory called `cities.py`. Put in this code:

```
import graph

# Create an empty graph
network = graph.Graph()

# Create city nodes and add to graph
long_beach = graph.Node("Long Beach")
network.add_node(long_beach)
los_angeles = graph.Node("Los Angeles")
network.add_node(los_angeles)
denver = graph.Node("Denver")
network.add_node(denver)
phoenix = graph.Node("Phoenix")
network.add_node(phoenix)
louisville = graph.Node("Louisville")
network.add_node(louisville)
cleveland = graph.Node("Cleveland")
network.add_node(cleveland)
boston = graph.Node("Boston")
network.add_node(boston)
nyc = graph.Node("New York City")
network.add_node(nyc)

# Create edges
graph.WeightedEdge(12, long_beach, los_angeles)
graph.WeightedEdge(23.0, los_angeles, denver)
graph.WeightedEdge(19, los_angeles, phoenix)
graph.WeightedEdge(52, phoenix, louisville)
graph.WeightedEdge(14, denver, louisville)
```

```
graph.WeightedEdge(80, phoenix, cleveland)
graph.WeightedEdge(9, denver, cleveland)
graph.WeightedEdge(8, cleveland, nyc)
graph.WeightedEdge(28, louisville, nyc)
graph.WeightedEdge(7, nyc, boston)
graph.WeightedEdge(13, cleveland, boston)

print(network)
```

Run it:

```
python3 cities.py
```

You should see some rather unexciting output:

```
(Graph:[(node:Long Beach, edges:1), (node:Los Angeles, edges:3), (node:Denver,
↪  edges:3),
(node:Phoenix, edges:3), (node:Louisville, edges:3), (node:Cleveland, edges:4),
(node:Boston, edges:2), (node:New York City, edges:3)])
```

But do not worry; we will make it more exciting in the next chapter!

This is a draft chapter from the Kontinua Project. Please see our website (https://kontinua.org/) for more details.

Appendix A

Answers to Exercises

Answer to Exercise 1 (on page 3)

The nodes in this graph are unique words. The edges represent a valid one letter operations that can transform one word into another word. These operations include adding a letter, removing a letter, or changing a letter into another. For example, the word `cat` would be connected to `cot` (change), `cart` (add), and `cut` (change), but not to `dog`. Usually this will be represented as a tree, which is a special type of graph we will cover later.

Here is an example of a word graph:

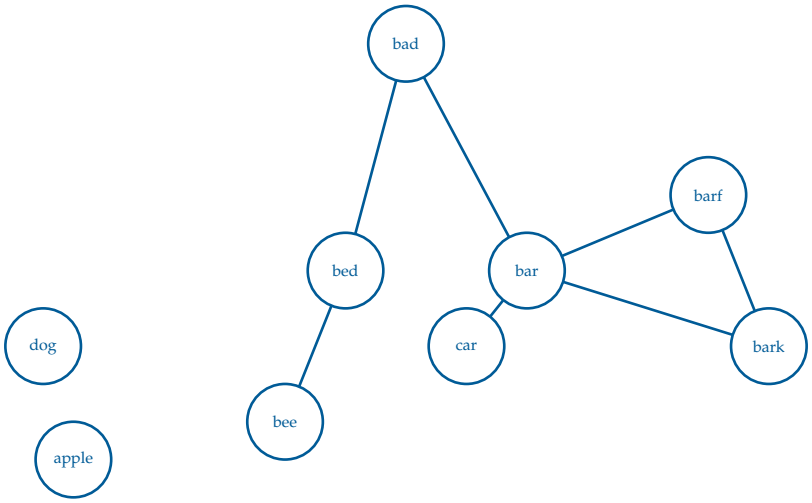


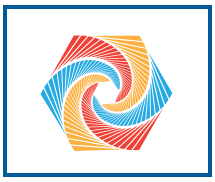
Figure 1.4: A word graph where the nodes are unique words and the edges represent a one letter operations between two words.

Answer to Exercise 2 (on page 5)

In a weighted graph, each node would have to store a set of neighbors and their respective weights. The best way to do this would be a tuple. For example,

- A: [(B, 4), (C, 3)]

- B: [(A, 4), (D, 8)]
- C: [(A, 3)]
- D: [(B, 8)]



Index

adjacency lists, 4

connected, 2
connected component, 2
connected components, 2
cycle, 2

degree, 2
directed, 1
directed graph, 1

edges, 1

graph, 1
    connected, 2
    directed, 1
    undirected, 1
Graph Databases, 4
Graph Theory, 2

in-degree, 4

neighbor, 4
nodes, 1

out-degree, 4

path, 2

subgraphs, 2

weighted edges, 4
weighted graph, 4